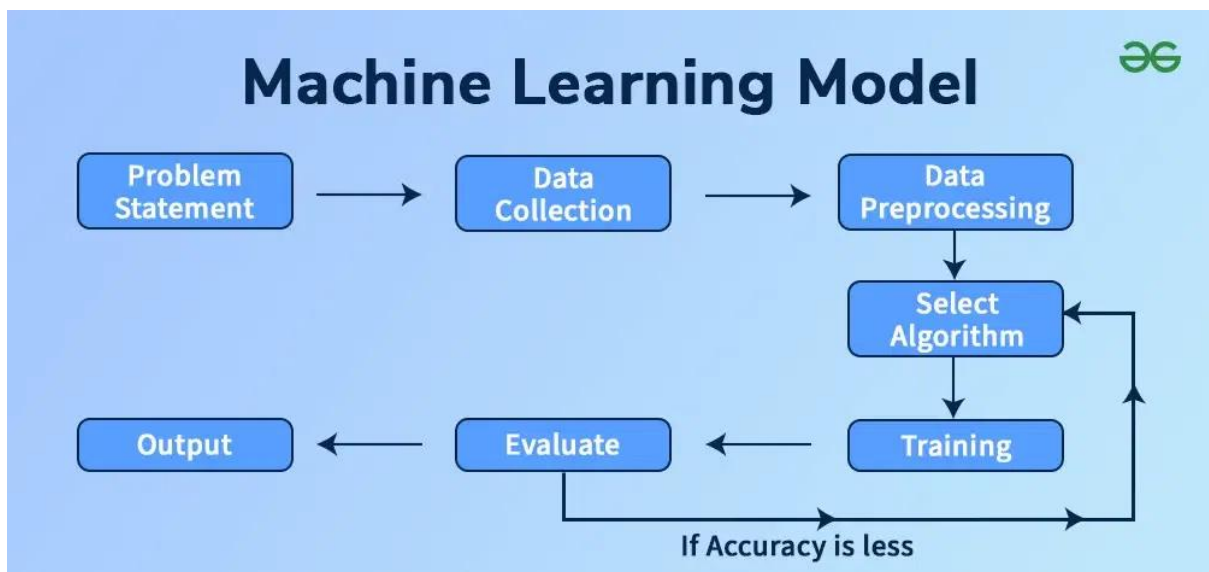


WIDS REPORT

STUDY OF MODELS

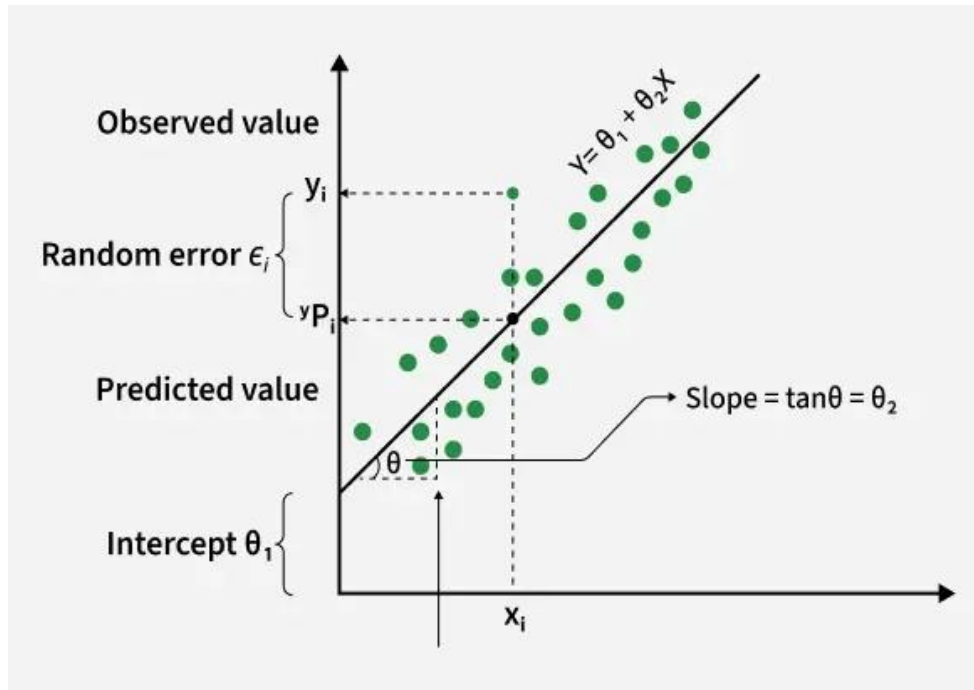


NAME: HARSHITH MANUPATI

ROLL NO: 24B1238

MENTOR NAME: Harsh Kumar Singh

Linear Regression:



Linear Regression is one of the fundamental and greatly used supervised learning algorithms. It is normally used in **prediction of a continuous target variable** based on one or many input features of the data. The core idea of linear reg is to get the relationship between the dependent variable and independent variables using a linear equation which gives the independent variables weights or parameters.

In a simple linear regression model, the model assumes a relationship of the given form which is a 2d line equation:

$$y = \beta_0 + \beta_1 x$$

where y is the output variable, x is the input feature, β_0 is the intercept, and β_1 is the slope.

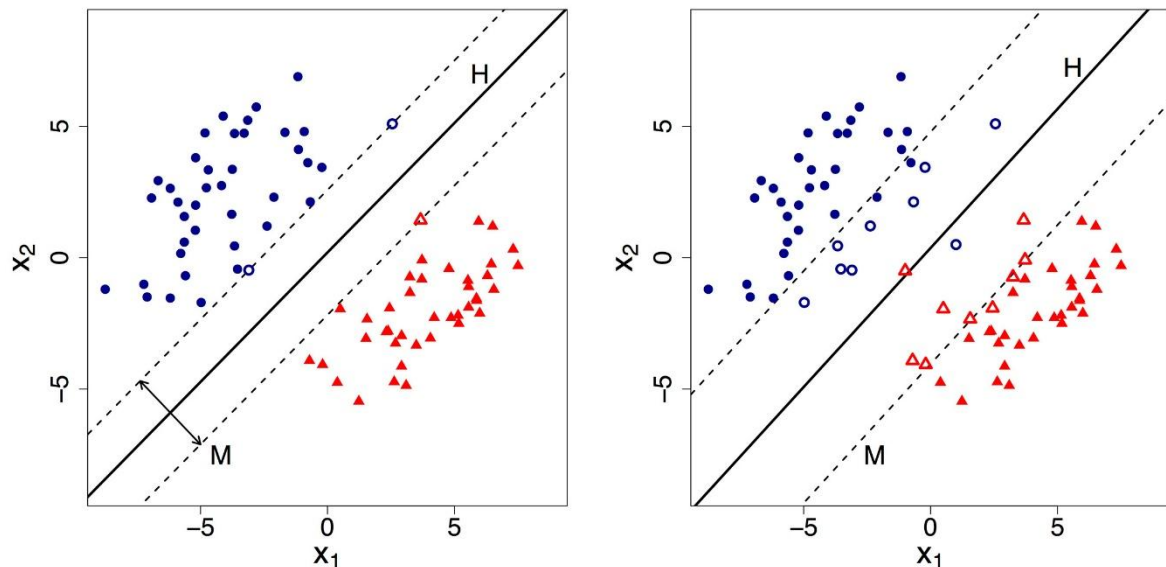
For multiple features, this extends to **multiple linear regressions**, where the model becomes a weighted sum of the given input features.

The main goal of linear regression is finding the best values of the weights that **minimizes the error** between predicted and actual values. This is commonly achieved using the function known as **Mean Squared Error (MSE)** loss function. Optimizing techniques like **Gradient Descent** or the **Normal Equation** are used estimating the weights or (parameters).

Linear regression is simple, interpretable, and computationally efficient. Generally, a start point who interested in data science or ml models. It provides insights in feature importance through the learned coefficients. However, it has many limitations. A major one is it assumes a linear relationship between features and targets and is sensitive to outliers(an observation which is more deviated), and can perform poorly when the relationship is highly non-linear or when multicollinearity exists among features.

Despite its simplicity, linear regression serves as a strong baseline model and is widely used in economics, finance, and scientific data analysis.

Support Vector Machines (SVMs)



Picture a tool that sorts things into groups or predicts values, one that works well even when there are many features involved. This method stands on solid math principles, handling complexity without getting confused by noise. It keeps performance steady across tasks like spotting faces in photos, sorting documents by topic, or finding patterns in genetic data. When dimensions grow large, it stays reliable, avoiding common traps of memorizing too much. Its reach extends into science and tech areas where precision matters most. Though built for two main jobs, its influence spreads far beyond basic uses.

Understanding Support Vector Machines

Finding the best dividing line sits at the heart of how SVM works. This divider, called a hyperplane, carves up points from distinct groups. When working on a flat grid, that separator takes the shape of a straight edge. As layers pile up beyond two dimensions, it stretches into a broad face or even twists into trickier forms. Though invisible to the eye, its role stays clear - keep categories apart.

Now picture this: when sorting things into two groups, SVM pushes the dividing line as far as possible from both sides. That space it creates? It's measured by how close the edge gets to the closest dots on either side. Usually, more room means fewer surprises later with new examples. Sometimes the gap matters most - wider often works smarter down the road.

Support Vectors

Closest dots near the line deciding groups go by the name support vectors. What matters most is how those spots pull the dividing plane into place. Not every point weighs in - just the key few shape where the border lands. Because it leans only on select examples, the method stores less yet handles messy inputs without breaking stride.

Maximum Margin Principle

What makes SVM stand out? It goes beyond just picking any dividing line. Rather than settling for separation alone, it hunts down the split where space between nearest opposite points stretches widest. That extra breathing room isn't just neat - it sharpens how reliably the model handles fresh examples later. The wider gap quietly fights against learning too much noise by accident.

Starts off looking at size - how small we can make the weights. What matters next shows up through rules that keep labels sorted right. Instead of just shrinking values, it balances against correct grouping. Ends with a tradeoff: tiny numbers versus accurate decisions.

Linearly Separable Versus Non Linearly Separable Data

When data points line up neatly, one flat boundary splits them without error. Yet most actual datasets come messy, tangled beyond clean division. Instead of failing here, support vector machines permit a few mistakes on purpose. These loose boundaries give breathing room for uncertainty. A tuning factor decides how much flexibility makes sense. Balance shifts depending on that setting - strictness versus tolerance.

Kernel Trick

Not straight? SVM shifts things around by using special math tools called kernels. These turn the setup into something more complex - where lines can finally split what once tangled together. The neat part: it skips heavy number-crunching by working through shortcuts baked into the method itself. No need to see every step - the result just appears like magic, but grounded in logic. Distance grows, yet effort stays low because nothing gets fully spelled out in raw form.

Kernel functions often seen in practice involve these examples

A straight line might do the job when things are neatly split apart - that's where a linear kernel steps in. It works best if the groups already sit on opposite sides of a mental ruler.

A curve's shape can be captured when data points follow a pattern. What matters is how the lines bend across space.

A favorite among many, the Radial Basis Function kernel works well when patterns twist and turn in tricky ways. Its strength shows up where straight-line guesses fail completely.

A curve-shaped pattern shows up in some learning systems, much like what you see when signals pass through network nodes.

How well an SVM works often hinges on which kernel you pick. Different problems respond better to different kernels. Picking one affects how boundaries form between data points. Some kernels handle curves; others stick to straight lines. The wrong fit might miss patterns entirely. Each option shapes the outcome in its own way. Performance shifts depending on this single element.

Hyperparameters in SVM

How well an SVM works comes down to how its settings are picked. Key among these are:

Starting with a high number here pushes accuracy on known data. That said, too much focus there risks poor performance later. Instead, lower values accept some mistakes early. This shift often helps when facing new cases. Balance matters most in the end.

One thing to note about the **RBF** kernel - its gamma value controls how much impact one training sample has across distances. How it spreads matters more than where it starts. Reaching further happens when gamma drops lower. A tighter grip on nearby points shows up with higher values. The effect fades faster if gamma climbs.

Getting the settings right matters a lot; people usually rely on methods like cross-validation to handle it.

Svm For Regression

Not just for sorting items, **SVM** handles prediction jobs too - this version goes by Support Vector Regression, or **SVR**. Instead of strict categorization, it draws a line through data points aiming to stay within a set distance from real values. Its strength shows when messy or unusual data pops up unexpectedly. Staying smooth matters as much as staying close. Outliers barely shake its results.

Advantages of SVM

SVM offers several advantages:

Effective in high-dimensional spaces.

Fewer samples? Still runs fine. Handles more features without trouble. Even if columns outnumber rows, it keeps going. Size doesn't stop it - works either way.

Fewer examples are kept during learning - just key ones that define boundaries. This keeps memory use low by focusing only on what shapes the decision line.

Performance holds up well across different cases. Solid backing comes from theory.

Limitations of SVM

Despite its strengths, **SVM** also has some limitations:

Heavy on processing when data piles up. Takes time to crunch if scale grows. Needs more power as size increases. Slows down with massive inputs. Works harder on bigger sets.

How well it works comes down to which kernel you pick, along with the settings fine-tuned alongside it.

Harder to understand than basic approaches such as decision trees or linear regression. Though straightforward methods show clear logic, this one hides its reasoning. Not every model reveals how it reaches conclusions. Simpler versions let you follow each step. This type stays foggy when explaining choices.

Applications of SVM

SVMs are widely used in many real-world applications, including:

Text and document classification

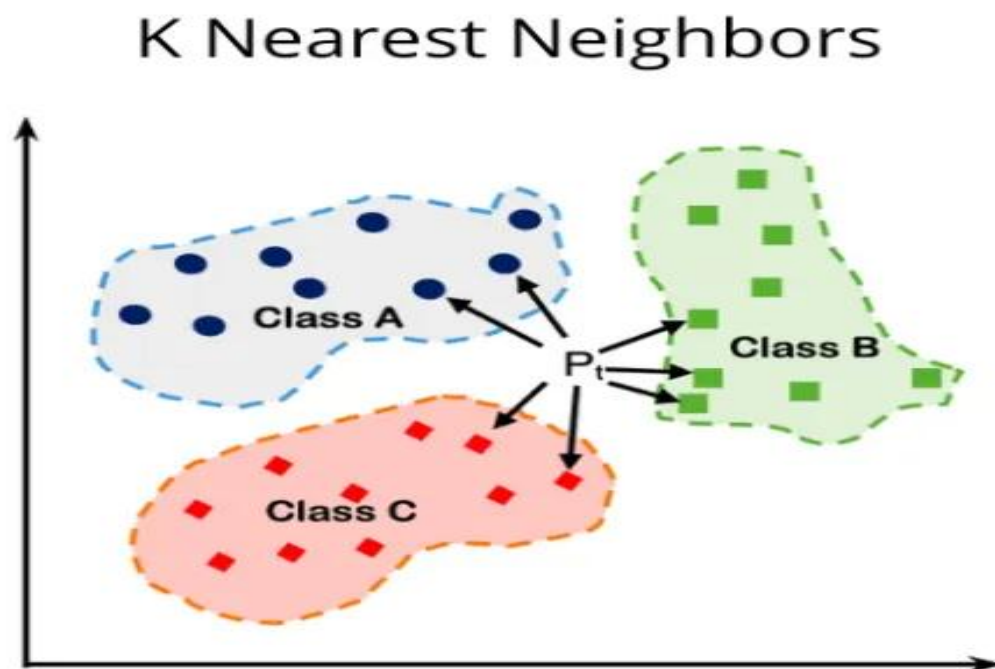
Image and face recognition

Handwritten digit recognition

Bioinformatics and gene classification

Fraud detection and spam filtering

K-Nearest Neighbours (KNN)



From up close, K-Nearest Neighbours looks basic but works well for sorting or estimating things using examples already seen. No math rules hidden underneath guide it - just patterns stored as they appear. Training doesn't build a summary; everything stays raw until prediction time. When guessing, it checks which past points sit closest and borrows their answers. Decisions come straight from neighbors, nothing smoothed out or simplified later. This method skips assumptions, keeps data untouched, relies only on similarity.

Simple things work well sometimes. KNN gets picked a lot at first because it does okay without much setup. When patterns in data are messy or hard to see, it still manages to help. Its strength shows up most when we do not know how inputs connect to outputs.

Basic Working Principle

Not like linear regression or decision trees, KNN skips the usual training step. Instead of learning patterns, it memorizes every data point during setup. Once a fresh example arrives, distances get measured to each stored case. The nearest K examples then guide the prediction by proximity.

A guess follows, shaped by what those nearby points show:

A vote decides the class label, based on which choice appears most often in classification tasks.

Folks often find the result in regression comes out as a sort of middle ground - shaped by nearby points, maybe some counting more than others

target values.

How KNN Measures Distance

How well KNN works mostly comes down to how we judge likeness among data entries. Distance formulas handle this task - Euclidean often leads, though others appear too

A line drawn through space between points defines Euclidean Distance - common when working with values that flow without breaks. This measure sticks close to how we see separation in the real world, using a clean path from one spot to another.

Going block by block, Manhattan Distance adds up steps on a straight-line grid. When you can only move like a rook on chessboards, this way of counting makes sense. Paths that zigzag through streets fit this measure best.

What hides behind the name Minkowski is a broader idea - it wraps up Euclidean, it takes in Manhattan. One formula, two familiar faces showing up in different light. Change one number, shift how space feels.

Facing forward instead of sideways - that's how cosine similarity works when sorting texts by direction rather than size. Direction stands out clearly even if lengths differ wildly across documents. Size fades into background while alignment takes center stage. Angles decide closeness here, not distance from origin. What points the same way feels closer, regardless of scale.

Picking one distance measure over another might change how well the model works, particularly if there are many dimensions involved.

How K Influences the Outcome

A value called K - how many close data points to look at - influences how the KNN method responds. What happens next depends heavily on that choice, shaping each outcome differently.

A tiny K - say, just one - lets random blips twist the outcome, pushing the model into overfitting territory.

A big K smooths out how choices are made, dampening noise effects - yet risks oversimplification that dulls accuracy. Still, too much smoothing might ignore key patterns simply because they seem rare.

Picking the right K matters a lot - usually figured out by trying different splits across data or testing what works in practice.

feature scaling and why it matters

Suddenly, picture how big numbers push small ones aside in KNN - scaling stops that shove. Without balance, one loud feature drowns out quieter traits during measurement. Steps shift sideways when units misalign, warping what matters. Distance bends where size shouts louder than signal.

Folks often adjust data ranges using methods like these

Min-Max Normalization

Standardization (Z-score normalization)

When scaling is off, KNN can still give flawed results despite a clean dataset. A single missing step throws everything sideways. Even neat data won't fix what's broken behind the scenes. Wrong scale means wrong distances - simple as that. Predictions drift when numbers aren't balanced early on.

KNN for Classification

A single test point gets labeled by looking at its closest matches - whichever group shows up most in those nearby points decides the outcome. When things that look alike usually belong together, this method tends to perform reliably.

Now here's a twist - closer points get extra weight in some KNN versions, making them count more than those far off. That little shift often sharpens the prediction, particularly if nearby examples simply matter more.

KNN for Regression

A number near the point helps shape the guess in KNN regressions, where nearby examples set the value. Starting from close ones, their targets get averaged, pulling the result toward familiar ground. Closer matches might count more if weights tilt the math their way. Influence fades with distance, like voices heard through fog - nearby shouts matter most.

When patterns in data twist and turn unpredictably, KNN regression often fits well.

Computational Complexity

A single guess can take ages when using KNN, especially if there are many examples to compare against. Instead of learning a shortcut ahead of time, it checks every past case each time someone asks for a result.

Funny thing, KNN keeps every bit of training data around, which means it gobbles up more memory than methods that boil things down into compact settings. Memory load climbs fast since nothing gets tossed or simplified along the way.

Advantages of KNN

KNN has several notable advantages:

Simple and easy to understand.

Ready to go right away.

Handles multiple classes by design. It just works that way.

Fits well with various ways to measure distance. Sometimes adjusts easily when the method changes. Works smoothly no matter the scale used. Handles shifts in measurement style without trouble.

Fine on modest data loads. Works well when information stays within manageable limits. Handles limited volumes without trouble. Scales poorly beyond that point.

Limitations of KNN

Even though it looks straightforward, KNN comes with a few drawbacks

High memory consumption.

Lasts longer when guessing big groups of data.

Fussiness around loud sounds shows up easily. Out-of-place points stick out just as much.

Suddenly, things slow down when dimensions grow too large. Hidden trouble shows up in vast data landscapes. Efficiency vanishes as complexity expands beyond control.

Getting it right means paying close attention to how features are scaled, while picking **K** needs a thoughtful approach. Each choice shifts the outcome in ways that aren't always obvious at first glance.

Applications of KNN

KNN is used in a variety of real-world applications, such as:

Pattern recognition

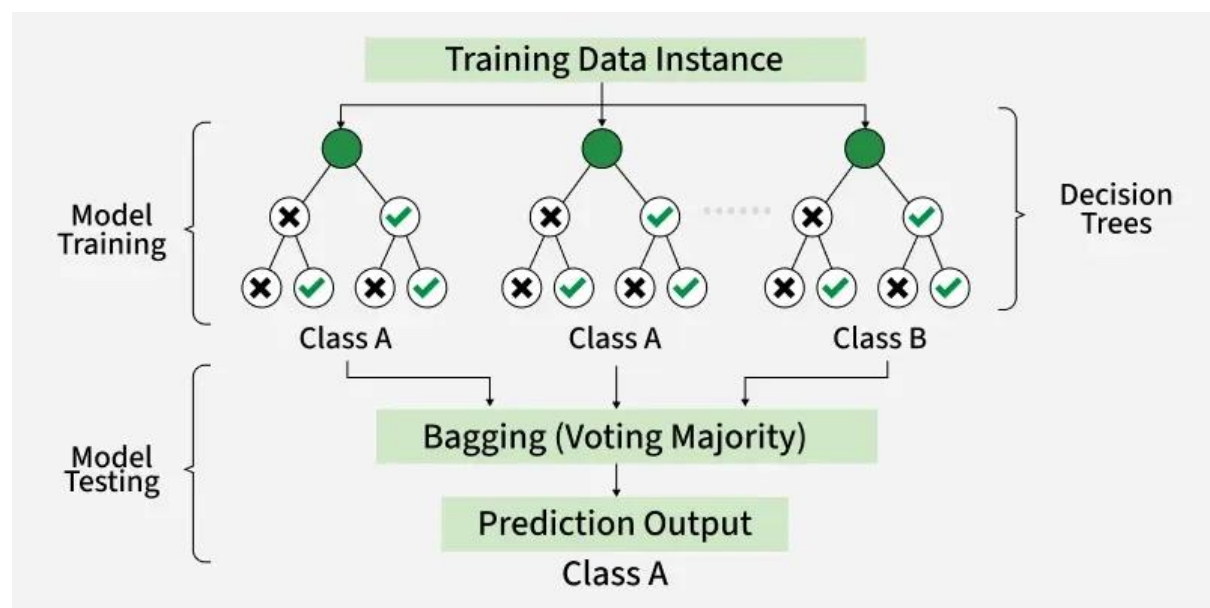
Recommendation systems

Handwritten digit recognition

Image and video classification

Predicting illnesses plus figuring out medical conditions

Random Forest



Random Forest is a powerful **ensemble learning algorithm** used for both **classification** and **regression** tasks. It is based on the idea of combining multiple models to produce a stronger and more reliable predictor. Instead of relying on a single decision tree, Random Forest builds a large collection of decision trees and aggregates their outputs to make the final prediction.

Random Forest is widely popular in both academic research and industry applications because of its **high accuracy, robustness to noise, and ability to handle complex datasets** with minimal parameter tuning.

Ensemble Learning Concept

The key principle behind Random Forest is **ensemble learning**, where multiple weak learners (decision trees) are combined to form a strong learner. A single decision tree is prone to overfitting, especially when it grows deep. Random Forest overcomes this limitation by averaging the results of many trees, thereby reducing variance and improving generalization performance.

This ensemble approach makes Random Forest much more stable and reliable compared to individual decision trees.

Construction of Random Forest

A Random Forest consists of many decision trees, each trained independently using randomness in two main ways:

1. Bootstrapping (Bagging)

Each decision tree is trained on a **random subset of the training data**, selected with replacement. This technique is known as **bootstrap aggregating (bagging)**. Since each tree sees a different subset of data, the trees become diverse, which improves the overall performance of the ensemble.

2. Random Feature Selection

At each split in a decision tree, only a **random subset of features** is considered instead of all available features. This prevents strong features from dominating the model and reduces correlation among trees.

Together, these two sources of randomness ensure that the trees are sufficiently different from each other, which is essential for reducing overfitting.

Prediction Mechanism

Once all the trees are trained, Random Forest makes predictions as follows:

- **Classification:** Each tree predicts a class label, and the final output is determined by **majority voting** among all trees.
- **Regression:** Each tree predicts a numerical value, and the final output is the **average** of all predictions.

This aggregation mechanism smooths out errors made by individual trees and leads to more accurate and stable predictions.

Reduction of Overfitting

One of the biggest advantages of Random Forest is its strong resistance to overfitting. While a single decision tree may perfectly fit the training data and perform poorly on unseen data, Random Forest reduces this risk by averaging many independent trees.

Because individual trees are trained on different data samples and feature subsets, their errors tend to cancel out, resulting in improved generalization.

Feature Importance

Random Forest provides a useful mechanism to estimate **feature importance**, which helps in understanding how much each feature contributes to the model's predictions. Feature importance can be calculated based on:

- Reduction in impurity (Gini impurity or entropy)
- Decrease in prediction accuracy when a feature is permuted

This makes Random Forest valuable not only for prediction but also for **feature selection and data analysis**.

Handling High-Dimensional and Large Datasets

Random Forest performs well on **high-dimensional datasets** and can handle large numbers of features without significant performance degradation. It can also handle both numerical and categorical data, and it is relatively robust to missing values and outliers.

These properties make it suitable for real-world datasets that are often noisy and complex.

Hyperparameters in Random Forest

Although Random Forest requires less tuning compared to many other models, some important hyperparameters include:

- **Number of trees (n_estimators):** More trees generally improve performance but increase computational cost.
- **Maximum depth of trees:** Controls how deep each tree can grow.
- **Minimum samples per split or leaf:** Helps control overfitting.
- **Number of features considered at each split:** Influences tree diversity.

Proper tuning can further improve performance, but even default settings often produce strong results.

Advantages of Random Forest

Random Forest offers several advantages:

- High accuracy and strong predictive performance
- Robust to noise and outliers
- Handles large and high-dimensional datasets well
- Reduces overfitting compared to decision trees
- Provides feature importance estimates
- Requires minimal preprocessing and tuning

Limitations of Random Forest

Despite its strengths, Random Forest has some limitations:

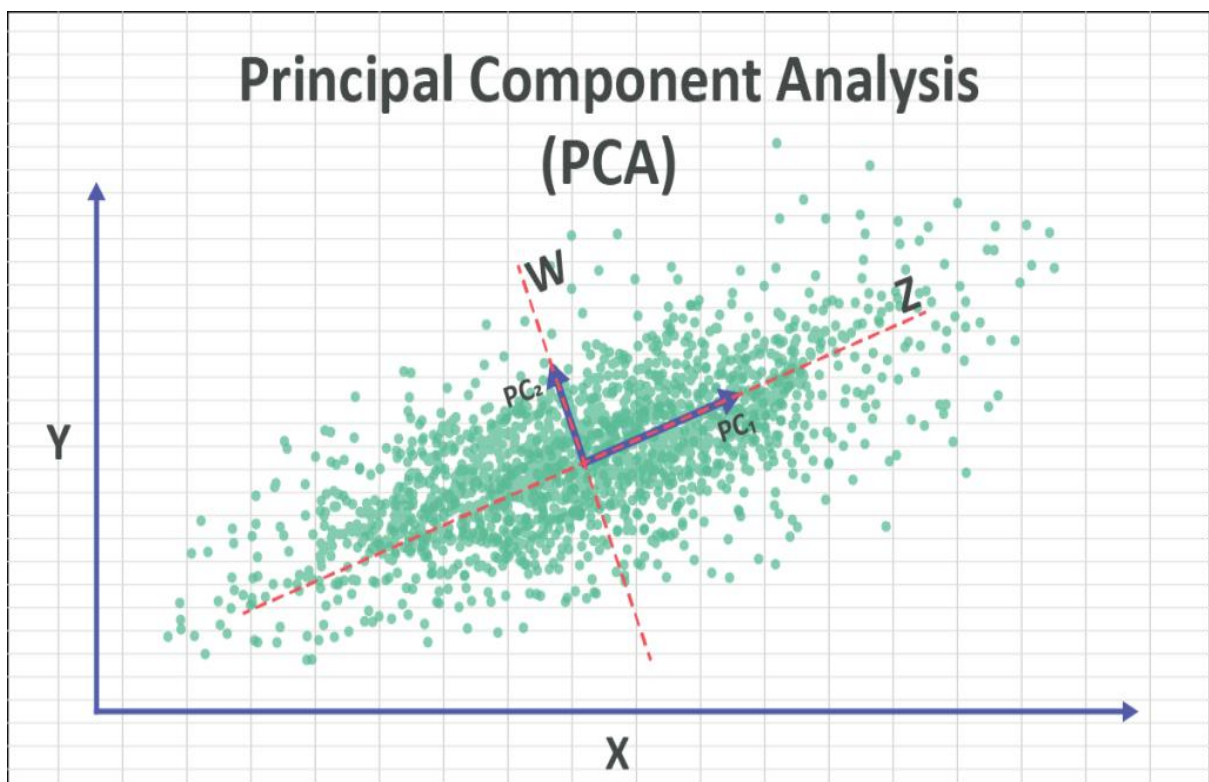
- Less interpretable than a single decision tree
- Computationally expensive for very large datasets
- Large memory usage due to storing many trees
- Slower prediction compared to simpler models

Applications of Random Forest

Random Forest is widely used in many domains, including:

- Credit scoring and risk analysis
- Medical diagnosis and disease prediction
- Fraud detection
- Image and speech recognition
- Customer segmentation and recommendation systems
- Bioinformatics and genomics

Principal Component Analysis (PCA)



Principal Component Analysis (PCA) is a widely used **unsupervised learning technique** for **dimensionality reduction**. Its primary goal is to simplify high-dimensional datasets by reducing the number of features while preserving as much of the original data's variability (information) as possible. PCA is especially useful when working with datasets that contain a large number of correlated features, which can make analysis and modelling computationally expensive and less effective.

Because PCA does not rely on labelled data, it is commonly used as a **preprocessing step** before applying machine learning algorithms.

Motivation for Dimensionality Reduction

In real-world applications, datasets often contain many features, some of which may be redundant, noisy, or highly correlated. High-dimensional data can lead to problems such as:

- Increased computational cost
- Overfitting in machine learning models
- Difficulty in visualization and interpretation
- The curse of dimensionality

PCA addresses these issues by transforming the data into a lower-dimensional space without losing significant information.

Core Idea of PCA

The central idea behind PCA is to transform the original set of features into a new set of variables called **principal components**. These principal components have the following properties:

- They are **linear combinations** of the original features.
- They are **uncorrelated (orthogonal)** to each other.
- They are ordered such that the first component captures the maximum possible variance, the second captures the next highest variance, and so on.

By selecting only the top few principal components, PCA effectively reduces the dimensionality of the dataset.

Principal Components and Variance

Each principal component represents a direction in the feature space along which the data varies the most. The amount of variance captured by a component indicates how much information it contains.

- **First principal component (PC1):** Captures the maximum variance in the data.
- **Second principal component (PC2):** Captures the maximum remaining variance while being orthogonal to PC1.
- Subsequent components follow the same principle.

Typically, only the first few components that explain a high percentage (e.g., 90–95%) of the total variance are retained, while the rest are discarded.

Mathematical Intuition Behind PCA

From a mathematical perspective, PCA is based on **eigenvalue decomposition** of the covariance matrix or **singular value decomposition (SVD)** of the data matrix. The eigenvectors represent the directions of the principal components, and the eigenvalues indicate the amount of variance explained by each component.

Although the mathematical details can be complex, the practical outcome is a transformation of data into a new coordinate system that emphasizes the most informative patterns.

Data Preprocessing for PCA

Before applying PCA, certain preprocessing steps are essential:

- **Mean centring:** Subtracting the mean from each feature.
- **Feature scaling:** Standardizing features to ensure equal contribution, especially when features have different units or ranges.

Without proper scaling, features with larger magnitudes may dominate the principal components.

Benefits of PCA

PCA offers several advantages in data analysis and machine learning:

- **Dimensionality reduction:** Reduces the number of features while retaining important information.
- **Noise reduction:** Eliminates components that capture low variance, which often represent noise.
- **Improved computational efficiency:** Faster training and prediction for machine learning models.
- **Visualization:** Enables visualization of high-dimensional data in 2D or 3D.
- **Reduction of multicollinearity:** Helps improve the performance of algorithms sensitive to correlated features.

PCA as a Preprocessing Step

PCA is frequently used before applying machine learning algorithms such as logistic regression, support vector machines, and clustering algorithms. By reducing dimensionality, PCA can improve model performance, stability, and generalization.

However, PCA should be applied carefully, as it does not consider class labels and focuses solely on variance.

Limitations of PCA

Despite its usefulness, PCA has certain limitations:

- **Loss of interpretability:** Principal components are combinations of original features, making them harder to interpret.
- **Linear method:** PCA captures only linear relationships and may not perform well for highly non-linear data.
- **Variance $\neq$ importance:** Features with low variance may still be important for prediction, but PCA may discard them.
- **Sensitive to scaling:** Improper scaling can lead to misleading results.

Applications of PCA

PCA is widely used across various domains, including:

- Image processing and face recognition
- Signal processing
- Bioinformatics and gene expression analysis
- Financial data analysis
- Exploratory data analysis and data visualization
- Data compression